# 🚀 Updated Step-by-Step Assignment 2 Guide for Sourav

**Name:** Sourav | **Roll No:** 51 | **GitHub:** RDSOURAV05 **Your Project Role:** Dataset Acquisition & Management

*Note: We are using the `software_engineering_lab_experiment_7` repository for this assignment's collaboration, while keeping the folder structures and code matching our main SAFE-VISION project.*

This document contains **every single step and exact code** you need. Follow it sequentially. Do not skip any terminal commands or GitHub website steps.

---

## Part 1: GitHub Issues & Branching (10 Marks)

### Step 1.1: Create an Issue on GitHub

1. Open your web browser and go to the new collaboration repository: `https://github.com/RDSOURAV05/software_engineering_lab_experiment_7`
2. Click on the **Issues** tab at the top.
3. Click the green **New Issue** button.
4. Title it: `Implement testing for Dataset Acquisition & Management`.
5. In the description, write: "Adding pytest unit tests and CI/CD for my module."
6. On the right side, click **Assignees** and click your own name (`RDSOURAV05`).
7. Click **Submit new issue**.

> 📸 ***SCREENSHOT 1 REQUIRED:*** *Take a screenshot of this issue page showing your title and your name assigned to it.*

### Step 1.2: Clone and Create a Branch (Terminal)

Open your computer's terminal (or Command Prompt / VS Code Terminal) and type these commands exactly:

```
# 1. Clone the collaboration repository to your local laptop
git clone https://github.com/RDSOURAV05/software_engineering_lab_experiment_7.git

# 2. Go into the project folder
cd software_engineering_lab_experiment_7

# 3. Create and switch to your own dedicated feature branch
git checkout -b feature-sourav
```

---

## Part 2: Writing the Code & Committing (15 Marks)

### Step 2.1: Add Your Unit Tests

*(Ensure your main project files for your module are present in this repo to test against.)* 1. Inside the `software_engineering_lab_experiment_7` folder, create a new file named `test_sourav.py`. 2. Open `test_sourav.py` in your code editor (like VS Code). 3. **Copy and paste the exact code below into `test_sourav.py` and save it:**

```
# Create this file in your project folder and name it: test_sourav.py

import pytest
from dataset_manager import load_dataset, split_data

# Test 1: Load valid dataset
def test_valid_dataset():
    df = load_dataset("data.csv")
    assert not df.empty

# Test 2: Missing file exception
def test_missing_file():
    with pytest.raises(FileNotFoundError):
        load_dataset("missing.csv")

# Test 3: Valid train/test split
def test_valid_split():
    train, test = split_data("data.csv", ratio=0.8)
    assert len(train) > len(test)

# Test 4: Split ratio exactly 1.0 (Boundary Value Analysis)
def test_boundary_split_ratio():
    with pytest.raises(ValueError):
        split_data("data.csv", ratio=1.0)

# Test 5: Invalid file extension (Equivalence Partitioning)
def test_invalid_extension():
    with pytest.raises(ValueError):
        load_dataset("data.txt")
```

### Step 2.2: Add the CI/CD Pipeline File (15 Marks)

The CI/CD pipeline tells GitHub to run your tests automatically. 1. In your project folder, create a folder named `.github`. 2. Inside `.github`, create a folder named `workflows` (Path so far: `.github/workflows`). 3. Inside `workflows`, create a file named `ci.yml` (Full path: `.github/workflows/ci.yml`). 4. **Copy and paste the exact code below into `ci.yml` and save it:**

```
name: CI
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-python@v6
        with:
          python-version: '3.11'
      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install pytest
          # pip install -r requirements.txt (Uncomment if you have a requirements file)
      - name: Run tests
        run: pytest -v
```

### Step 2.3: Commit and Push Your Code

Go back to your terminal and type:

```
# 1. Add all the new files to git
git add .

# 2. Make your first commit
git commit -m "Added unit tests for Dataset Acquisition & Management"

# 3. (Required for marks) Make a second commit.
# Open README.md, add a space at the end of any line, save it, and run:
git add README.md
git commit -m "Updated README for feature-sourav"

# 4. Push your branch to GitHub
git push -u origin feature-sourav
```

---

## Part 3: Pull Requests & GitHub Actions (15 Marks)

### Step 3.1: Create a Pull Request (PR)

1. Go back to your GitHub repository (`software_engineering_lab_experiment_7`) in your web browser.
2. You will see a yellow banner saying `feature-sourav had recent pushes`. Click **Compare & pull request**.

3. Title it: `Add tests and CI for Sourav`.
4. On the right side, click **Reviewers** and select one of your teammates.
5. Click **Create pull request**.
6. **Ask your teammate to go to GitHub, view your PR, and click "Approve".**

> 📸 ***SCREENSHOT 2 REQUIRED:*** *Take a screenshot of the Pull Request page after your teammate has approved it.*

### Step 3.2: Check GitHub Actions (CI/CD)

1. On GitHub, click the **Actions** tab at the top.
2. You should see your recent push running the "CI" workflow. Wait a minute for it to finish.
3. Once it turns into a **Green Tick** ✅, click on it.

> 📸 ***SCREENSHOT 4 REQUIRED:*** *Take a screenshot showing the successful green tick for the Actions run.*

### Step 3.3: Add the Status Badge

1. In the Actions tab where you see your successful workflow, click the `...` menu on the top right.
2. Select **Create status badge**.
3. Click **Copy status badge Markdown**.
4. Go to your repository's `README.md` file, edit it, and paste this markdown code at the very top. Save and commit.

> 📸 ***SCREENSHOT 5 REQUIRED:*** *Take a screenshot of the main repo page showing the README with the passing status badge.*

---

## Part 4: Project Management Requirements (10 Marks)

### Step 4.1: Kanban Board

1. On GitHub, click the **Projects** tab. Create a project if one does not exist.
2. Create three columns: `To Do`, `In Progress`, `Done`.

3. Add the issue you created in Step 1.1 to the board and drag it into the **In Progress** column.

> 📸 ***SCREENSHOT 3 REQUIRED:*** *Take a screenshot of the Kanban board showing your issue in the In Progress column.*

### Step 4.2: Gantt Chart & Risks (To include in your final PDF)

You must include a Gantt chart and a risk list in your submitted PDF. **Gantt Chart Tasks (Draw this as a simple table or bar chart in Word):** * Task 1: Requirement Analysis & Repo Setup * Task 2: Dataset Preparation & Preprocessing * Task 3: Model Development (Dataset Acquisition & Management) * Task 4: Module Integration & Unit Testing * Task 5: CI/CD Pipeline Setup * Task 6: Final Deployment & Documentation

**Risk List (Copy-paste exactly as is into your PDF):** 1. **Risk 1 (Technical):** Inaccurate AI classification. *Mitigation:* Use a diverse dataset and adjust risk score thresholds. 2. **Risk 2 (Operational):** GitHub merge conflicts. *Mitigation:* Commit frequently and always peer-review pull requests. 3. **Risk 3 (Technical):** CI/CD pipeline failures due to dependencies. *Mitigation:* Pin specific package versions in `requirements.txt`.

### Step 4.3: Test Case Table (To include in your final PDF)

Copy this table into your final report document (Word/Docs):

| TC ID | Function Tested | Input | Expected Output | Actual Output | Pass/ Fail |
|---|---|---|---|---|---|
| TC-01 | load_dataset | data.csv | DataFrame | (Leave Blank) | (Leave Blank) |
| TC-02 | load_dataset | missing.csv | FileNotFoundError | (Leave Blank) | (Leave Blank) |
| TC-03 | split_data | ratio=0.8 | len(train) > len(test) | (Leave Blank) | (Leave Blank) |
| TC-04 | split_data | ratio=1.0 | ValueError (BVA) | (Leave Blank) | (Leave Blank) |
| TC-05 | load_dataset | data.txt | ValueError (EP) | (Leave Blank) | (Leave Blank) |
| *(Note: BVA = Boundary Value Analysis, EP = Equivalence Partitioning)* | | | | | |

### Step 4.4: Deployment Note (To include in your final PDF)

Copy this exact paragraph into your final report document:

> *"The SAFE-VISION project can be deployed on an Infrastructure as a Service (IaaS) platform such as AWS EC2, or a Platform as a Service (PaaS) such as Heroku. Because the technology stack is Python-based (FastAPI and Streamlit), the application can be seamlessly packaged as a Docker container by configuring a Dockerfile, ensuring a consistent execution environment from local testing to cloud production."*

---

## 🎯 FINAL SUBMISSION CHECKLIST (Due: 19/10/2026)

Create a single Word Document, add all the items below, and save it as a PDF named exactly: **A2_51_Sourav.pdf**

- [ ] Screenshot 1: Assigned Issue
- [ ] Screenshot 2: Approved Pull Request
- [ ] Screenshot 3: Kanban Board
- [ ] Screenshot 4: Successful GitHub Actions run (Green Tick)
- [ ] Screenshot 5: README file showing the Status Badge
- [ ] The Gantt Chart (6 Tasks)
- [ ] The Risk List (3 Risks with mitigations)
- [ ] The Test Case Table (from Step 4.3)
- [ ] The Deployment Note (from Step 4.4)

*Submit this PDF on Google Classroom.*